

DistillWheel 详细设计

 **核心结论：** DistillWheel 不是一个“把 teacher 输出拿来 SFT student”的单点工具，而是一套面向垂直任务的能力蒸馏与数据飞轮系统。它的核心闭环是：任务定义 -> 数据生成 -> 质量门禁 -> 蒸馏训练 -> 领域评测 -> 发布门禁 -> 失败样本回流。

系统第一阶段重点跑通黑盒蒸馏和数据飞轮 MVP：先支持 teacher 生成/修复、规则与 verifier 校验、Raw Signal Store、Artifact Builder、SFT Artifact、ms-swift/TRL 训练、领域评测和失败样本回流。白盒 KD、OPD、Verifier-Guided RL 作为后续算法扩展，Agent/GUI 作为后续 vertical extension。

从分享页补充后的关键边界是：DistillWheel 主要处在 SFT、Preference/RL、领域后训练和数据飞轮层，不是预训练数据平台；它提供的是 task-specific data production、verifier 和 recipe orchestration。

1. 系统要解决什么问题

DistillWheel 的目标是把一个具体垂直任务，从“零散 prompt 和脚本”变成可复现、可迭代、可评测的小模型生产流程。它关注的不是单个蒸馏算法，而是把任务定义、数据、训练、评测和回流全部串起来。

不做什么：

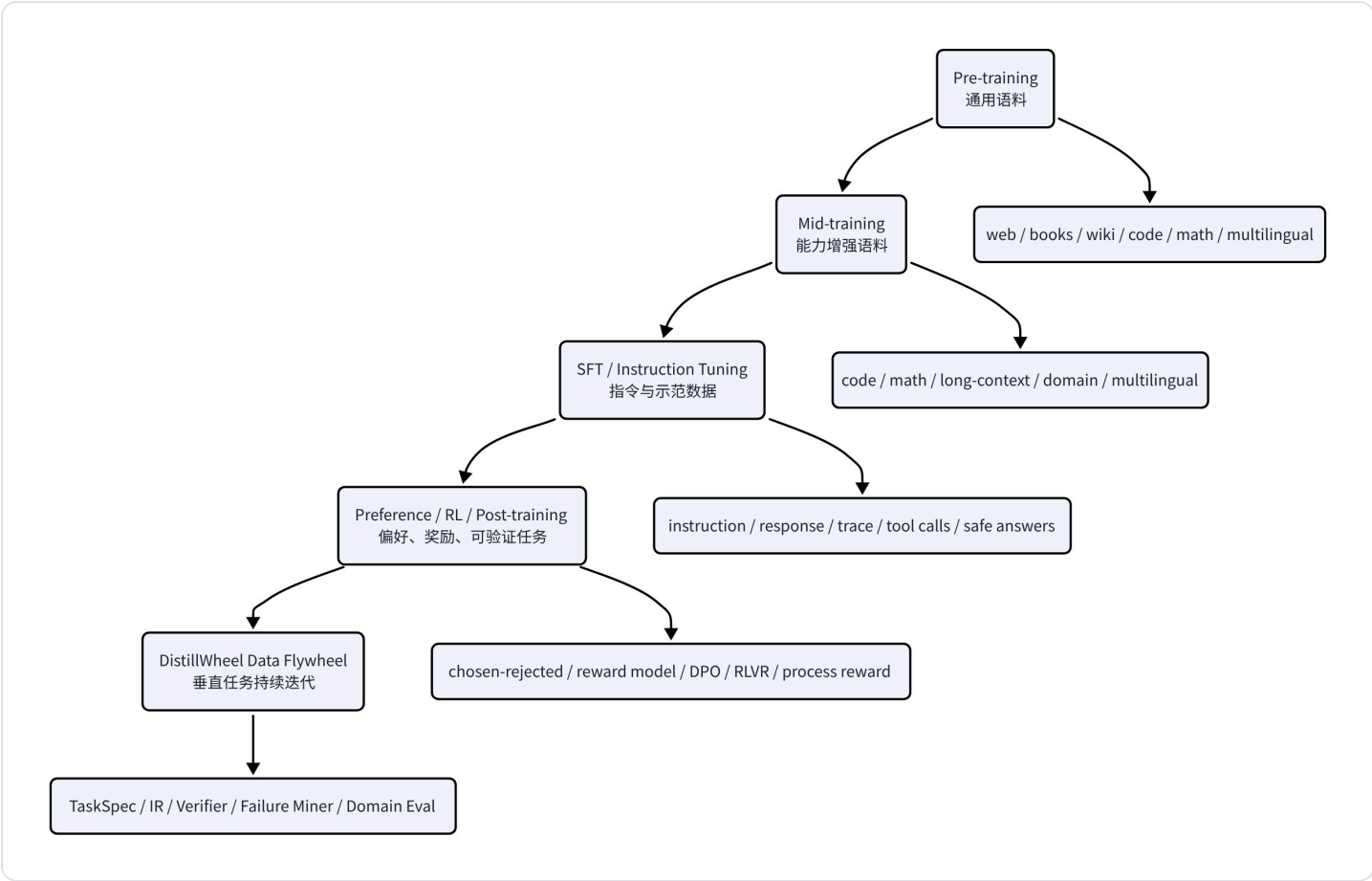
- 不是只把 10 个 seed 题 paraphrase 成 1000 个题。
- 不是只用一个通用 LLM Judge 判断所有任务。
- 不是重新实现所有 trainer、serving 和数据平台。
- 不是把 EasyDistill、KDFlow、TRL、ms-swift 简单包一层。

要做什么：

- 把垂直任务标准化成 TaskSpec 和 IR Schema。
- 用 source-grounded、concept-grounded、failure-driven 的方式生成数据。
- 用任务专属 verifier 做质量门禁和纠错。
- 把 SFT、DPO、GKD、RLVR 等训练策略封装成 recipe。
- 让失败样本持续回流，形成 guarded data flywheel。

2. DistillWheel 在模型训练数据栈中的位置

通用大模型的数据工程通常分为四层：预训练、继续预训练、SFT/Instruction Tuning、Preference/RL/Post-training。DistillWheel 不从头建设预训练语料平台，而是专注在垂直任务的 SFT、偏好优化、Verifier-driven RL 和失败样本回流。



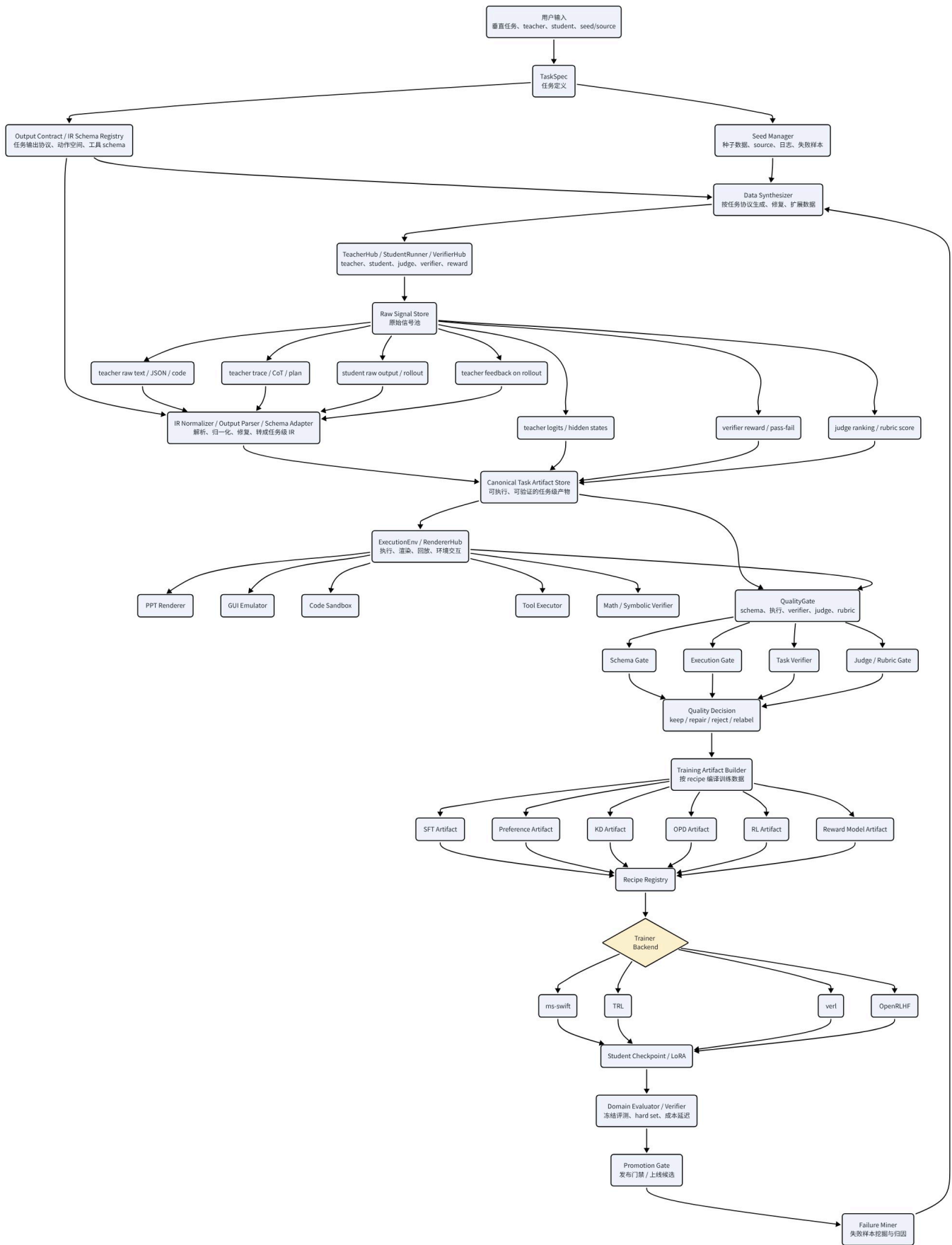
公开模型	公开数据规模/配比	关键做法	对 DistillWheel 的启发
Llama 3	约 15T tokens；公开资料中给出约 50% general knowledge、25% math/reasoning、17% code、8% multilingual 的 mix 方向。	用分类器和 scaling-law 实验调 data mix；post-training 采用多轮 SFT 和 DPO，并持续收集新偏好数据和 SFT 数据。	数据不是越多越好，而是按能力目标分层配比；DistillWheel 也要记录每轮数据来源、能力标签和训练贡献。
Qwen3	约 36T tokens 级别的大规模预训练数据。	覆盖多语言和多领域；用视觉语言模型从 PDF 抽取文本，并用数学/代码专家模型生成合成数据。	source-grounded generation 很重要。DistillWheel 需要支持从 PDF、网页、代码仓库、教材等 source 中构造任务。

DeepSeek-V3	约 14.8T 高质量、多样 tokens。	提升数学和编程样本比例，同时扩展多语言覆盖。	垂直能力增强必须显式建模能力分布。DistillWheel 的 TaskSpec 要能声明 subskill 和目标覆盖率。
-------------	------------------------	------------------------	--

❗ **边界说明：**如果目标是“训练一个通用基础模型”，那需要预训练和继续预训练数据工程；如果目标是“把 teacher 的某个垂直能力压到 student”，DistillWheel 主要工作在 SFT、偏好数据、verifier reward、评测和失败回流层。

3. 总体架构

DistillWheel 的训练前数据链路需要拆成三层对象：**Raw Signal 原始信号**、**Canonical Task Artifact 任务级标准产物**、**Training Artifact 训练算法专用格式**。Teacher / Student / Verifier 最初产生的不是训练数据，而是原始信号；IR Normalizer 先把可执行输出整理成任务级 IR；QualityGate 结合执行环境和 verifier 判断是否可用；Training Artifact Builder 再按 SFT / DPO / KD / OPD / RL 等 recipe 编译成训练框架能吃的数据格式。



💡 **设计原则：**Recipe 是算法逻辑，Backend 是执行框架。同一个 DPO recipe 可以用 ms-swift 或 TRL 跑；同一个 OPD recipe 可以用 verl 或 TRL 自定义 trainer 跑。KDFlow、EasyDistill

更适合作为竞品/参考项目或专用蒸馏 runtime，不放在通用 Trainer Backend 主干里。

4. 核心模块设计

核心模块按系统流程划分，不按算法名称划分。算法进入系统前，必须先明确它需要哪类 artifact，再由 Recipe Registry 选择合适 backend 执行。

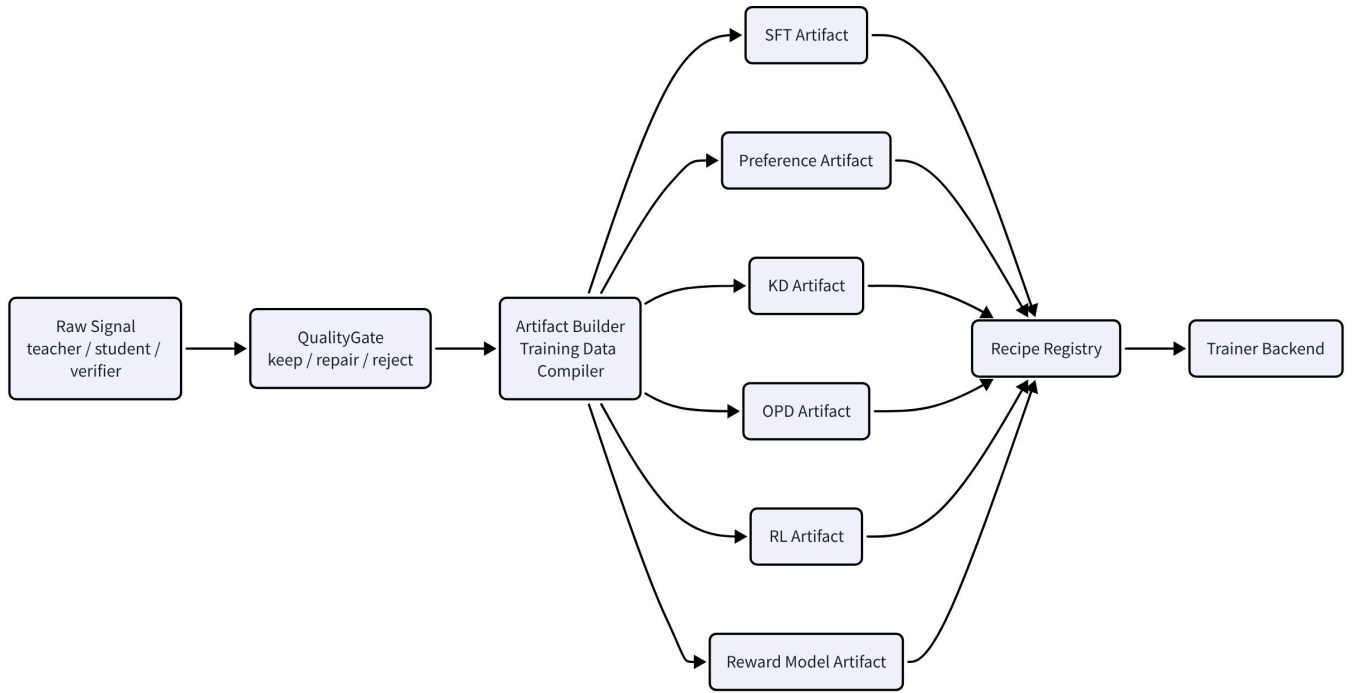
模块	职责	主要输入/输出	允许用户定制什么
TaskSpec	把垂直任务变成可执行配置，包括输入类型、输出协议、teacher、student、recipe、eval。	输入 task.yaml；输出标准化任务对象。	任务名称、输入字段、输出目标、指标、数据源、预算限制。
IR / Output Contract	定义 student/teacher 必须输出什么格式，使结果可解析、可执行、可校验。	输入 JSON Schema/grammar；输出结构化 IR、动作协议、工具 schema。	JSON Schema、字段约束、渲染协议、动作空间、工具调用格式。
Seed Manager	管理 seed 数据、source 文档、用户日志、历史失败样本和每轮数据版本。	输入 seed/source/log；输出 dataset_version、lineage。	上传 seed、导入文档、指定 split、冻结 eval set、设置数据血缘规则。
Data Synthesizer	从 seed、source、概念图和失败样本生成候选任务、候选输出或修复请求。	输入任务定义和数据源；输出 candidate samples。	生成模式、难度范围、覆盖目标、样本数量、prompt 模板。
TeacherHub / VerifierHub	统一接入 API teacher、本地 teacher、VLM teacher、judge、verifier、reward function。	输出 answer、trace、logits、hidden states、rollout feedback、reward、rubric score。	teacher 列表、调用参数、license 策略、verifier 插件、reward 函数、成本上限。
Raw Signal Store	统一保存 teacher、student、verifier 产生的原始信号，不提前假设它们只会变成 SFT 或 DPO。	输入原始信号；输出带 lineage 的 raw signal records。	信号类型、存储格式、保留字段、压缩策略、数据血缘。
QualityGate	判断信号/样本是否可信。它不负责决定用 SFT、OPD 还是 RL。	输入 raw signal；输出 keep/repair/reject、score、error_type。	规则校验器、执行器、PRM、VLM judge、阈值、抽检比例。
Artifact Builder	把通过质量门禁的信号编译成不同 recipe 需要的数据形	输出 SFT、Preference、KD、OPD、RL、RM	artifact schema、字段映射、采样策略、hard

	态。	artifact。	negative 构造、版本规则。
Preference Builder	Artifact Builder 的偏好数据子模块，专门构造 chosen/rejected、ranking、score。	输入 bad output、repair output、judge score；输出 Preference / RM artifact。	chosen/rejected 来源、pair 构造规则、ranking 策略、rubric。
Recipe Registry	声明要跑什么训练算法，以及该算法需要哪些 artifact 和 backend candidates。	输入 artifact、model、recipe config；输出训练任务配置。	recipe 类型、required_artifacts、loss、超参、backend、LoRA/全参策略。
Trainer Backend	执行训练。Backend 是实现框架，不是算法 taxonomy。	输入训练任务配置；输出 checkpoint/LoRA。	ms-swift、TRL、verl、OpenRLHF、GPU 配置、保存策略。
Domain Evaluator	用领域指标证明 student 是否真的有用，不只看通用 benchmark。	输入 checkpoint 和 frozen eval；输出评测报告。	指标、验证器、回归集、hard set、人工抽检规则。
Promotion Gate	判断模型是否可以作为上线候选或进入下一轮训练。	输入评测报告和成本延迟指标；输出 promote/reject。	发布阈值、成本目标、延迟目标、失败容忍度、回滚条件。
Failure Miner	把 student 错误样本结构化，形成下一轮 teacher repair、hard case generation 或 verifier-driven training 入口。	输入失败样本；输出 failure_type、severity、repair_target。	错误分类、采样优先级、回流比例、是否进入 hard set。

4.1 Training Artifact Builder：训练数据编译器

 **核心定义：**Artifact Builder 负责把 Teacher / Student / Verifier 产生的 Raw Signal 编译成不同 Recipe 能消费的训练数据格式。它不是训练算法，也不是训练后端，而是 Raw Signal 和 Recipe 之间的训练数据编译层。

DistillWheel 不能假设所有 teacher 数据最后都只是 SFT 数据或 DPO 数据。不同 recipe 需要的数据结构完全不同：SFT 需要 prompt 和 completion，DPO 需要 chosen/rejected，Logit KD 需要 teacher logits，OPD 需要 student rollout 上的 teacher feedback，GRPO/RLVR 需要 verifier 或 reward function。



Builder	输入 Raw Signal	输出 Artifact	服务的 Recipe
SFTArtifactBuilder	teacher final output、 teacher reasoning trace、 teacher repair output。	<code>prompt</code> 、 <code>completion</code> ， 可选 <code>reasoning_trace</code> 。	SFT、Trace SFT、 Response KD、 Failure-repair SFT。
PreferenceArtifactBuilder	teacher 修复版、student 错误版、 judge ranking、 rubric score。	<code>prompt</code> 、 <code>chosen</code> 、 <code>rejected</code> ，或 <code>ranking</code> / <code>score labels</code> 。	DPO、SimPO、 KTO、Reward Model。
KDArtifactBuilder	teacher logits、top-k logprobs、hidden states、 attention / intermediate representations。	<code>input_ids</code> 、 <code>teacher_distribution</code> 、 <code>teacher_hidden</code> 。	Logit KD、Hidden- state KD、 MiniLLM、 DistiLLM。
OPDArtifactBuilder	student rollout、teacher feedback on rollout、 token/step-level supervision。	<code>prompt</code> 、 <code>rollout</code> 、 <code>teacher_feedback_on_rollout</code> 。	GKD、OPD、 SOD、Agent step- wise distillation。
RLArtifactBuilder	prompt set、verifier、 reward function、 environment、rollout config。	<code>prompt</code> 、 <code>reward_fn</code> 、 <code>verifier</code> 、 <code>rollout_config</code> 。	GRPO、PPO、 RLVR。通常不是严格 KD，但属于后训练 闭环。
RewardModelArtifactBuilder	多个输出、pairwise label、 rubric score、judge score。	<code>ranking</code> 、 <code>score labels</code> 、 <code>pairwise labels</code> 。	Reward Model、 reranker、judge

！ 边界：QualityGate 判断信号是否可信；Artifact Builder 决定这些可信信号应该编译成什么训练数据格式；Recipe Registry 决定使用哪种训练目标；Trainer Backend 只负责执行训练。

一句话规则：Raw Signal 决定系统有什么信息；Artifact Builder 决定这些信息变成什么训练数据；Recipe 决定用什么目标函数；Backend 决定用什么框架跑。

Recipe 对 Artifact 的声明示例

```
1  recipe: opd_v1
2  required_artifacts:
3    - opd_artifact
4  artifact_schema:
5    prompt: string
6    student_rollout: list
7    teacher_feedback_on_rollout: list
8  backend_candidates:
9    - verl
10   - trl_custom
```

4.1.1 参考项目怎么分层

💡 判断：目前没有一个开源 repo 完整覆盖 DistillWheel 需要的 Artifact Builder。可参考项目要按能力拆开看：数据格式、数据生成与反馈、KD/OPD/RL runtime、Artifact Store 分别参考不同项目。

Artifact Builder 的关键不是简单做格式转换，而是把 raw teacher / student / verifier signal 编译成 recipe-specific artifact。因此参考 repo 不能只看 trainer，也要看 dataset format、feedback pipeline、runtime data protocol 和 artifact lineage。

第一类：数据格式与 Trainer 输入协议

项目	链接	最值得参考的部分	DistillWheel 用法
----	----	----------	-----------------

TRL	https://github.com/huggingface/trl	SFT、DPO、Reward、GKD 等 trainer 的 dataset format；尤其是 prompt / completion、prompt / chosen / rejected。	定义 SFT Artifact、Preference Artifact、Reward Model Artifact 的 schema。
Axolotl	https://github.com/axolotl-ai-cloud/axolotl	Dataset format handler、preprocess、prompt template、conversation template、preference 数据入口。	参考 dataset config 和 formatter 组织方式，做可配置 artifact builder。
ms-swift	https://github.com/modelscope/ms-swift	工程化 SFT、DPO、GRPO、多模态训练的数据入口和配置方式。	MVP 阶段优先适配的训练后端之一，反推 artifact 输出格式。

第二类：数据生成、AI feedback 与偏好构造

项目	链接	最值得参考的部分	DistillWheel 用法
distilabel	https://github.com/argilla-io/distilabel	LLM generation、judge、scoring、filtering、preference data generation 的 pipeline DAG。	参考 Data Synthesizer、QualityGate、Preference Builder 的流水线连接方式。
EasyDistill	https://github.com/modelscope/easydistill	black-box / white-box KD、data synthesis、SFT、ranking optimization、RL recipe 的组织方式。	作为竞品和 KD toolkit 参考；不放在通用 Trainer Backend 主干里。

第三类：KD / OPD / RL 运行时协议

项目	链接	最值得参考的部分	DistillWheel 用法
KDFlow	https://github.com/songmzhang/KDFlow	KD Artifact、OPD Artifact、cross-tokenizer artifact、teacher/student 解耦、hidden-state transfer。	后期白盒/灰盒 KD 和 OPD 的专用 runtime 参考；不是通用 Trainer Backend 主干。
verl	https://github.com/verl-project/verl	DataProto、rollout batch、reward batch、	参考 OPD、GRPO、RLVR 的 RL Artifact /

		actor/critic/ref/reward model 之间的数据协议。	OPD Artifact 内部协议。
OpenRLHF	https://github.com/OpenRLHF/OpenRLHF	SFT、Reward Model、DPO、PPO/RLHF、rejection sampling 的训练数据流。	参考 RLHF Artifact、Reward Model Artifact、RL Artifact 的组织方式。

第四类：Artifact Store 与 Lineage

项目	链接	最值得参考的部分	DistillWheel 用法
W&B Artifacts	https://docs.wandb.ai/models/artifacts	数据集、模型、文件和 metadata 的版本化、血缘追踪和实验关联。	作为研究阶段 artifact store 的参考。
DVC	https://github.com/iterative/dvc	Git-like 数据版本控制，管理数据集、模型文件和 pipeline 产物。	适合管理 seed、raw signal、artifact、eval report 的版本。
MLflow	https://github.com/mlflow/mlflow	Experiment tracking、model registry、artifact logging。	适合记录每轮飞轮的 recipe、指标、checkpoint 和模型注册。

4.1.2 两个核心编译案例

案例	Raw Signal	Artifact Builder 输出	后续 Recipe
Teacher repair	prompt、student_bad_output、teacher_repaired_output、judge_score、error_type。	一份 SFT Artifact: prompt / repaired completion；一份 Preference Artifact: chosen = repaired, rejected = bad output。	Failure-repair SFT；后续可接 DPO / SimPO / RM。
Student rollout	prompt、student_rollout、teacher_feedback_on_rollout、verifier_reward、pass/fail。	一份 OPD Artifact: rollout + teacher feedback；一份 RL Artifact: prompt + verifier / reward_fn / rollout_config。	OPD / GKD / SOD；或 GRPO / PPO / RLVR。

案例 A: repair 信号编译成 SFT + Pr...

```
1  {
```

案例 B: rollout 信号编译成 OPD + RL

```
1  {
2    "prompt": "解一道数学题",
```

```
2  "prompt": "把 README 转成 PPT  
   IR",  
3  "student_bad_output": "布局重叠  
   的 IR",  
4  "teacher_repaired_output": "修  
   复后的 IR",  
5  "artifacts": ["sft",  
   "preference"]  
6  }
```

```
3  "student_rollout": ["step1",  
   "step2"],  
4  "teacher_feedback": "第 2 步推导  
   错误",  
5  "verifier_reward": 0,  
6  "artifacts": ["opd", "rl"]  
7  }
```

4.1.3 实现路线

阶段	优先实现	主要参考	完成标准
第一阶段	SFTArtifactBuilder、PreferenceArtifactBuilder、基础 artifact lineage。	TRL、Axolotl、distilabel、W&B / DVC。	teacher final output / repair output 能稳定编译成 SFT 和 Preference 数据。
第二阶段	KDArtifactBuilder、OPDArtifactBuilder、RLArtifactBuilder、RewardModelArtifactBuilder。	EasyDistill、KDFlow、verl、OpenRLHF。	同一批 raw signal 能按 recipe 编译成 KD / OPD / RL / RM artifact。
第三阶段	多模态 artifact、Agent trajectory artifact、跨 tokenizer 对齐 artifact。	KDFlow、verl、任务专属 verifier。	支持 GUI / Agent / VLM 场景的 trajectory、bbox、tool trace、reward artifact。

5. 数据模型与任务配置

系统必须从第一天保存完整数据血缘，否则飞轮跑几轮后无法复现，也无法判断性能提升来自哪一类数据。

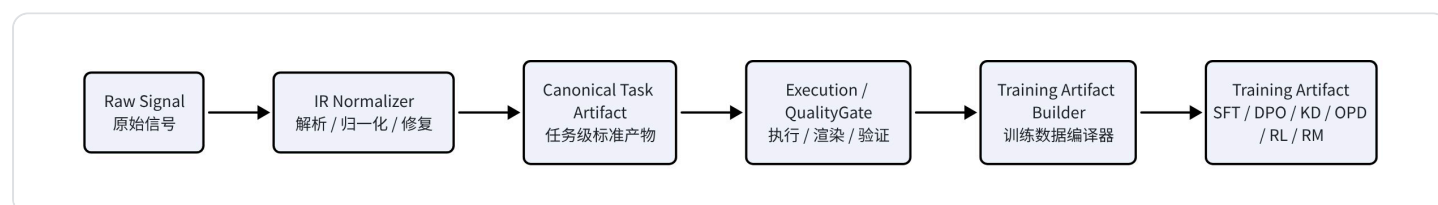
5.1 三层数据对象：Raw Signal、Canonical Task Artifact、Training Artifact

 **核心结论：**Teacher / Student / Verifier 最初产生的不是训练数据，而是 Raw Signal。系统必须先把可执行输出整理成符合任务协议的 Canonical Task Artifact，再由 Training Artifact

Builder 编译成 SFT / DPO / KD / OPD / RL 等训练算法能吃的 Training Artifact。

这三层对象必须分开，否则系统会把“可执行输出”和“训练输入格式”混为一谈。可执行 IR 确实是 artifact，但它是任务级 artifact，不是所有训练算法都能直接消费的训练 artifact。

层级	是什么	典型内容	主要用途
Raw Signal 原始信号	teacher、student、verifier、judge、环境最初吐出来的原始数据。它可以不干净、不稳定、不符合 schema。	自然语言、Markdown、JSON 片段、代码、logits、hidden states、student rollout、verifier score、judge ranking。	保留原始证据和 lineage；后续由 parser、normalizer、repair、quality gate 使用。
Canonical Task Artifact 任务级标准产物	经过解析、归一化、修复后，符合某个 Output Contract / IR Schema 的任务产物。它应该可解析、可执行、可验证。	MathSolutionIR、GUIActionIR、PptRenderIR、TrajectoryIR、PatchIR、ClaimCitationIR。	用于执行、渲染、环境交互、质量检测、失败归因、评测。
Training Artifact 训练算法专用格式	按具体 recipe 编译出来的训练输入。它不一定可执行，但必须符合 trainer / recipe 的数据协议。	SFT prompt-completion、DPO chosen/rejected、KD logits / hidden、OPD rollout + feedback、RL prompt + reward_fn。	直接交给 ms-swift、TRL、verl、OpenRLHF 等 Trainer Backend。



5.2 Output Contract / IR Schema Registry：任务输出协议

Output Contract / IR Schema 是模型输出和下游程序之间的接口协议。它不是“所有模型都必须输出 JSON”的意思，而是要求进入后续执行、评测、训练链路的有效样本，最终能被整理成一种可检查、可执行、可训练的标准格式。

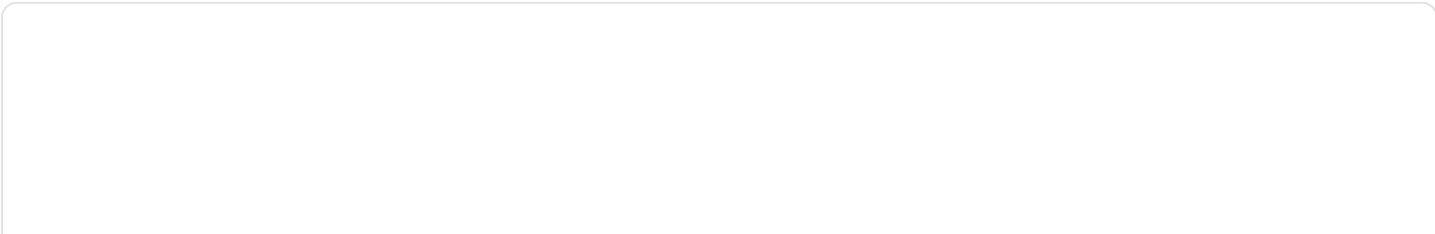
！ 准确表述：Teacher 原始输出可以不符合 IR Schema；但如果这个样本要进入 DistillWheel 后续链路，就必须被 parser / normalizer / repair / constrained decoding 整理成符合某个 Output Contract 的 Canonical Task Artifact。整理失败的样本要么重新生成，要么丢弃。

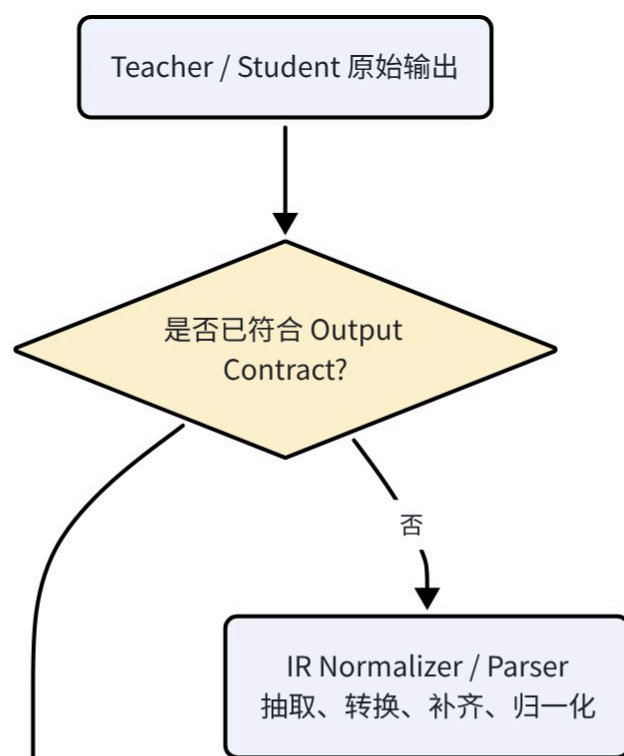
协议形态	适用任务	能检查什么	典型工具 / 参考
------	------	-------	-----------

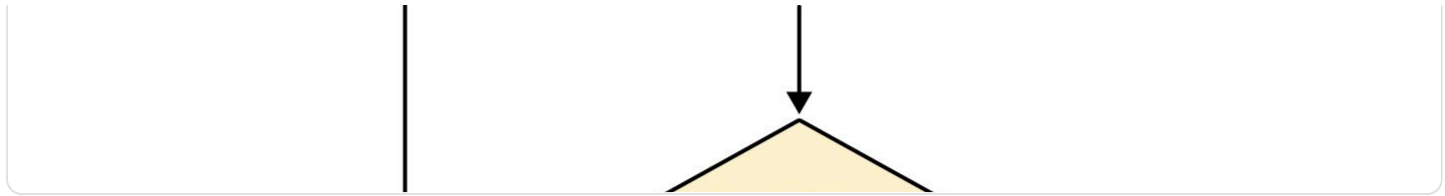
JSON / JSON Schema	数学解题、PPT IR、GUI action、Agent tool call、结构化抽取。	字段是否存在、类型是否正确、枚举是否合法、数值范围是否合法。	JSON Schema、OpenAI Structured Outputs、Outlines、XGrammar。
函数调用 / Tool Schema	工具调用 Agent、Deep Research、搜索/计算/代码执行类任务。	tool_name 是否存在、arguments 是否满足类型和必填字段、是否可回放。	MCP、OpenAI tool calling、LangGraph / Agent tool schema。
代码 DSL / Program IR	Code2PPT、图表生成、代码生成、数据分析。	是否能 parse、是否能执行、是否调用受限 API、是否生成目标文件。	PptxGenJS、python-pptx、AST、SQL parser、sandbox。
坐标 / BBox / Action	GUI grounding、GUI Agent、VLM action prediction。	坐标是否越界、bbox 是否合法、action 类型是否合法、能否在环境中执行。	ScreenSpot-Pro、AndroidWorld、OSWorld。
XML / YAML / Markdown 子集	文档生成、报告生成、配置生成、结构化内容编辑。	标签是否闭合、层级是否合法、字段是否完整、是否能渲染。	XML parser、YAML parser、Markdown AST。

5.2.1 Teacher 输出的三种模式

模式	Teacher 原始输出	系统处理方式	适用场景
A. 直接符合 IR	Teacher 直接按 JSON Schema、函数调用、bbox/action 或代码 DSL 输出。	直接进入 schema validation 和 QualityGate。	强模型、结构简单、API 支持 structured output 的任务。
B. 自然语言转 IR	Teacher 先输出自然语言答案、解释、计划、推理过程。	IR Normalizer 抽取字段、归一化格式、补齐缺失字段，再验证 schema。	数学、报告、研究、解释性任务。
C. 输出代码 / 文件 / 轨迹	Teacher 输出 PptxGenJS 代码、patch、tool trajectory、SQL、LaTeX 等。	系统把它包装成 Code IR / Trajectory IR，并在 sandbox / renderer / executor 中验证。	Code2PPT、代码生成、Agent、GUI、工具调用。







5.3 IR Normalizer / Output Parser / Schema Adapter

IR Normalizer 是 Raw Signal 到 Canonical Task Artifact 的转换层。它不负责训练，也不负责最终打分；它负责把不稳定的模型输出变成稳定的任务级标准产物。

职责	具体动作	输出
Parse	从自然语言、Markdown、代码块、tool trace 中抽取结构化字段。	初步结构化对象。
Normalize	统一字段名、坐标系、单位、枚举值、答案类型、路径格式、工具名。	符合当前任务约定的字段形式。
Repair	修复 JSON 尾逗号、缺字段、非法枚举、坐标越界、代码块缺失、tool arguments 格式错误。	可再次验证的候选 IR。
Validate	用 Output Contract / IR Schema 做结构验证。验证不通过就进入 teacher repair 或 reject。	Canonical Task Artifact 或 error report。
Adapt	把上游 schema 转成下游执行器需要的 schema，例如 Semantic IR 转 Render IR，Render IR 转 Code IR。	可执行 / 可渲染 / 可回放的任务产物。

5.4 Canonical Task Artifact 和 Training Artifact 的区别

!

最容易混淆的点：QualityGate 之后确实已经有 artifact，但它是 Canonical Task Artifact。它能被执行、渲染、验证，却不等于能被 SFT / DPO / OPD / GRPO 等训练算法消费。训练前仍然需要 Training Artifact Builder。

对比项	Canonical Task Artifact	Training Artifact
回答的问题	这个任务输出能不能被系统理解、执行、渲染、验证？	这个样本能不能被某个训练 recipe / trainer 直接消费？
面向对象	Renderer、executor、environment、verifier、evaluator、failure miner。	SFTTrainer、DPOTrainer、KD loss、OPD trainer、GRPO / PPO runner、

		RM trainer。
典型形态	MathSolutionIR、PptRenderIR、GUIActionIR、ToolTrajectoryIR、PatchIR。	prompt-completion、prompt-chosen-rejected、input-teacher_logits、rollout-feedback、prompt-reward_fn。
是否可执行	通常应该可执行、可渲染、可回放或可验证。	不一定可执行；它重点是匹配训练算法数据协议。
由谁产生	IR Normalizer / Output Parser / Schema Adapter。	Training Artifact Builder。

5.4.1 同一个 PPT Render IR 可以编译成多种训练数据

目标 Recipe	Training Artifact	为什么不能直接用 Render IR
SFT	prompt + completion。completion 可以通过 QualityGate 的 Render IR。	Trainer 需要输入文本或 messages 格式，不能只拿一个孤立 IR。
DPO / SimPO	prompt + chosen + rejected。chosen 是好 IR，rejected 是失败 IR。	Preference 训练需要成对偏好，不是单个可执行产物。
RLVR / GRPO	prompt + reward_fn + verifier_config + rollout config。	RL 训练需要采样和 reward，不是离线 completion。
OPD / GKD	student_rollout + teacher_feedback_on_rollout。	OPD 学的是 student 分布上的 teacher feedback，不是普通 SFT 数据。

TaskSpec 示例

Sample 统一格式

```
task.yaml
1  task_name: code_to_ppt
2  input_types:
3    - markdown
4    - code
5    - csv
6  output_contracts:
7    semantic_ir: ppt_semantic_ir_v1
8    render_ir: ppt_render_ir_v1
9    code_ir: pptxgenjs_program_v1
10 teachers:
11   generation_teacher: qwen_max
12   repair_teacher: local_qwen_72b
13 student:
```

```
sample.json
1  {
2    "sample_id": "code2ppt_000123",
3    "task_name": "code_to_ppt",
4    "input": {
5      "markdown": "...",
6      "tables": [],
7      "code_files": []
8    },
9    "raw_signals": {
10     "teacher_raw_output": "...",
11     "student_raw_output": "...",
12     "judge_scores": [],
13     "verifier_results": []
14   }
```

```

14     base_model: qwen3_4b
15     trainable_adapter: lora
16     normalizers:
17         - markdown_to_semantic_ir
18         - semantic_to_render_ir
19         - render_ir_to_code_ir
20     quality_gates:
21         - schema_gate
22         - render_success_gate
23         - layout_geometry_gate
24         - content_faithfulness_gate
25     artifact_builders:
26         - sft_artifact_builder
27         - preference_artifact_builder
28     recipes:
29         - api_sft_v1
30         - failure_repair_sft_v1
31         - dpo_quality_v1
32     eval:
33         - frozen_eval
34         - hard_failure_set
35         - cost_latency

```

```

14     },
15     "canonical_task_artifacts": {
16         "semantic_ir": {},
17         "render_ir": {},
18         "code_ir": {}
19     },
20     "execution_results": {
21         "pptx_rendered": true,
22         "slide_images": [],
23         "layout_errors": []
24     },
25     "quality_decision": {
26         "action": "keep",
27         "score": 0.91,
28         "failure_types": []
29     },
30     "training_artifacts": {
31         "sft":
32         "artifacts/sft/code2ppt_000123.js
33         onl",
34         "preference":
35         "artifacts/dpo/code2ppt_000123.js
36         onl"
37     },
38     "lineage": {
39         "dataset_version": "v003",
40         "teacher": "local_qwen_72b",
41         "output_contract_version":
42         "ppt_render_ir_v1",
43         "quality_gate_version":
44         "ppt_quality_v2"
45     }
46 }

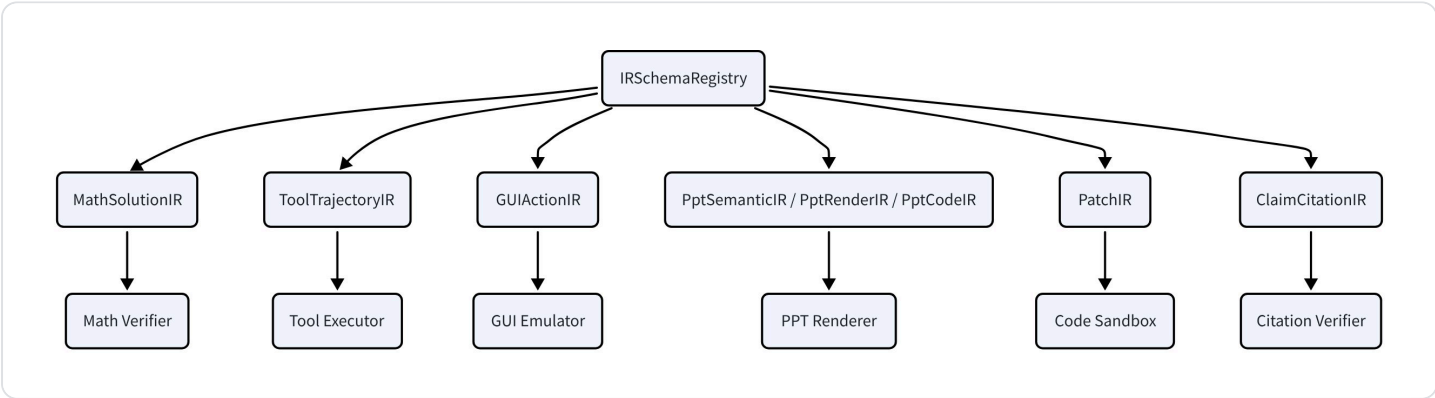
```

5.5 不同任务的 Output Contract / IR Schema 设计

 **核心原则：**DistillWheel 只有统一的 Output Contract 接口，没有统一的万能 IR。数学、代码、PPT、GUI、Agent、Research 的“正确输出”完全不同，因此每个 vertical 都要定义自己的任务级协议。

Output Contract 的目标不是把所有任务都压成一种 JSON，而是让每个任务的输出都能被程序稳定处理。它至少要回答四个问题：输出长什么样、怎么检查格式、怎么执行或回放、怎么把失败原因结构化。

任务	Canonical Task Artifact	为什么需要 IR	后续能做什么
数学推理	MathSolutionIR : steps、final_answer、 answer_type	最终答案要能抽取，步骤要能 定位错误，答案类型要能验 证。	exact match、SymPy、 PRM、错步修复、trace SFT。
工具调用 Agent	TrajectoryIR : thought、tool_call、 observation、final	自然语言轨迹无法稳定回放， 必须知道每一步调用了哪个工 具和参数。	工具执行、轨迹回放、 trajectory distillation、 Agent RL。
GUI Grounding	GUIActionIR : bbox、 point、action、 coordinate_space	模型输出必须落到屏幕坐标或 控件区域，否则无法执行点击 和判断命中。	bbox hit、IoU、click accuracy、环境状态验 证。
PPT 生成	PptSemanticIR 、 PptRenderIR 、 PptCodeIR	只输出大纲不够，必须有可渲 染的元素位置、样式、层级和 代码调用。	PPTX 渲染、布局检查、 VLM 评审、SFT/DPO/RL artifact。
代码生成	PatchIR : file_path、 diff、tests、run_command	代码输出必须知道改了哪些文 件、如何运行测试、失败日志 属于哪一步。	编译、单测、静态检查、 repair pair、execution feedback。
Deep Research	ClaimCitationIR : claims、evidence、 citations、confidence	研究报告不能只看文风，必须 知道每个 claim 被哪些 citation 支持。	citation checking、 factuality verifier、 claim-level repair。



5.5.1 数学任务：MathSolutionIR

数学任务的 IR 不应该只保存最终答案。只保存 final answer 会让系统无法判断 student 是“格式错”“算术错”“建模错”还是“中间步骤错”。所以数学 IR 至少要包含题目、步骤、最终答案和答案类型。

```
math_solution_ir.json
1  {
2    "problem_id": "math_001",
```

字段	作用
solution_steps	用于 trace SFT、过程检查、

```

3   "problem": "小明有 3 个苹果，又买了 2 个，一共有几个？",
4   "solution_steps": [
5     {"step_id": 1, "statement": "已有 3 个苹果。"},
6     {"step_id": 2, "statement": "又买了 2 个，所以计算 3 + 2。"},
7     {"step_id": 3, "statement": "3 + 2 = 5。"}
8   ],
9   "final_answer": "5",
10  "answer_type": "integer",
11  "verification": {
12    "method": "exact_match",
13    "expected": "5"
14  }
15 }

```

final_answer	用于最终答案验证和自动打分
answer_type	区分 integer、decimal、exp
verification	声明用 exact match、nume

！ 设计要点：数学能力的 QualityGate 优先用可验证工具，不要一上来依赖 LLM Judge。简单题用 exact match，表达式题用 symbolic equivalence，复杂推理再引入 PRM 或 step-level verifier。

5.5.2 工具调用 Agent: TrajectoryIR

Agent 蒸馏的核心不是“让模型写一段看起来像思考的文本”，而是让模型学会在多轮环境里选择工具、传参、读取 observation、继续决策。TrajectoryIR 要保存每一步的 observation、action、tool result 和 final answer。

```

tool_trajectory_ir.json
1  {
2    "task_id": "tool_001",
3    "user_request": "查找论文 A 的年份，并计算距今多少年。",
4    "trajectory": [
5      {
6        "step": 1,
7        "thought": "需要先搜索论文 A 的发表年份。",
8        "action": {
9          "type": "tool_call",
10         "tool_name": "search",
11         "arguments": {"query": "paper A publication year"}
12       },
13       "observation": "论文 A 发表于 2021 年。"
14     ],

```

```
15     {
16         "step": 2,
17         "thought": "当前年份是 2026 年，相差 5 年。",
18         "action": {
19             "type": "final_answer",
20             "answer": "论文 A 发表于 2021 年，距今 5 年。"
21         }
22     }
23 ]
24 }
```

检查点	具体检查	失败后怎么回流
Tool Schema	tool_name 是否存在，arguments 是否满足工具 schema。	生成 tool argument repair 样本。
Execution	工具能不能执行，返回值是否为空，是否超时。	把错误日志和 observation 交给 teacher 修复。
Grounding	final answer 是否被 observation 支持。	生成 claim-evidence 对或 rejected trajectory。
Efficiency	是否绕路、重复调用、调用无关工具。	构造 chosen/rejected 轨迹偏好数据。

5.5.3 GUI 任务：GUIActionIR

GUI 任务的输出不能只是一句“点击设置”。系统要能真的把动作执行到屏幕上，因此 IR 要明确动作类型、坐标、坐标系和可选 bbox。这样 QualityGate 才能判断是否命中目标控件。

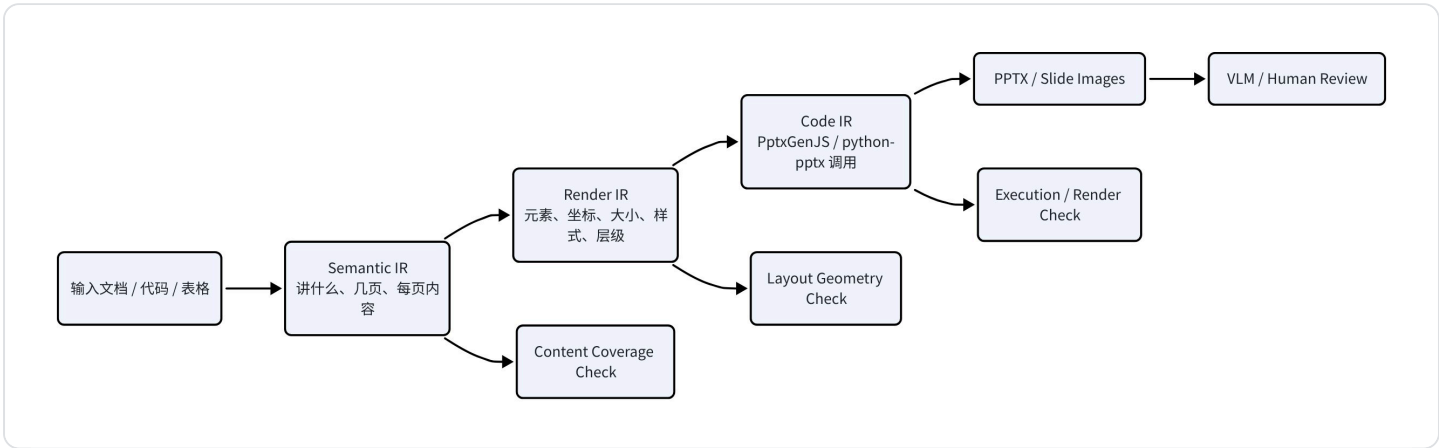
```
gui_action_ir.json
1  {
2      "instruction": "点击右上角设置按钮",
3      "screen_id": "screen_0007",
4      "grounding": {
5          "bbox": [0.82, 0.04, 0.95,
6                  0.12],
7          "point": [0.885, 0.08],
8          "coordinate_space":
9              "normalized"
10     },
11     "action": {
12         "type": "click",
13         "x": 0.885,
14         "y": 0.08
```

Verifier	判断内容
坐标范围	x/y 是否在合法范围，坐标系
BBox Hit	click point 是否落在目标 bbo
环境状态	点击后页面状态是否发生预期
轨迹质量	多步任务里是否绕路、误触、

```
13     }
14 }
```

5.5.4 PPT 任务：Semantic IR、Render IR、Code IR 三层协议

PPT 任务不能只定义一层 IR。只定义大纲会缺少布局信息；只定义坐标又容易丢失语义结构；只输出代码则难以做内容级评估和局部修复。因此 Code2PPT 建议拆成三层：Semantic IR、Render IR、Code IR。



层级	主要字段	能解决的问题	典型错误
Semantic IR	slide_title、sections、bullets、data_refs、speaker_intent。	保证讲什么、页数、内容覆盖、逻辑顺序。	主题缺失、内容编造、页间逻辑断裂。
Render IR	elements、x/y/w/h、font_size、color、z_index、layout。	保证元素位置、大小、层级、可读性。	越界、重叠、字体过小、图表位置错误。
Code IR	PptxGenJS 或 python-pptx 调用、asset path、render command。	保证能生成 PPTX，并能复现渲染过程。	代码运行失败、资源路径错、生成文件打不开。

```
ppt_render_ir.json
1  {
2    "slide_id": "slide_01",
3    "canvas": {"width": 13.333, "height": 7.5, "unit": "inch"},
4    "elements": [
5      {
6        "id": "title_1",
7        "type": "text",
8        "role": "title",
9        "text": "项目背景",
```

```
10         "x": 0.6,
11         "y": 0.4,
12         "w": 12.0,
13         "h": 0.6,
14         "font_size": 30,
15         "z_index": 10
16     }
17 ]
18 }
```

5.5.5 代码与研究任务：PatchIR 和 ClaimCitationIR

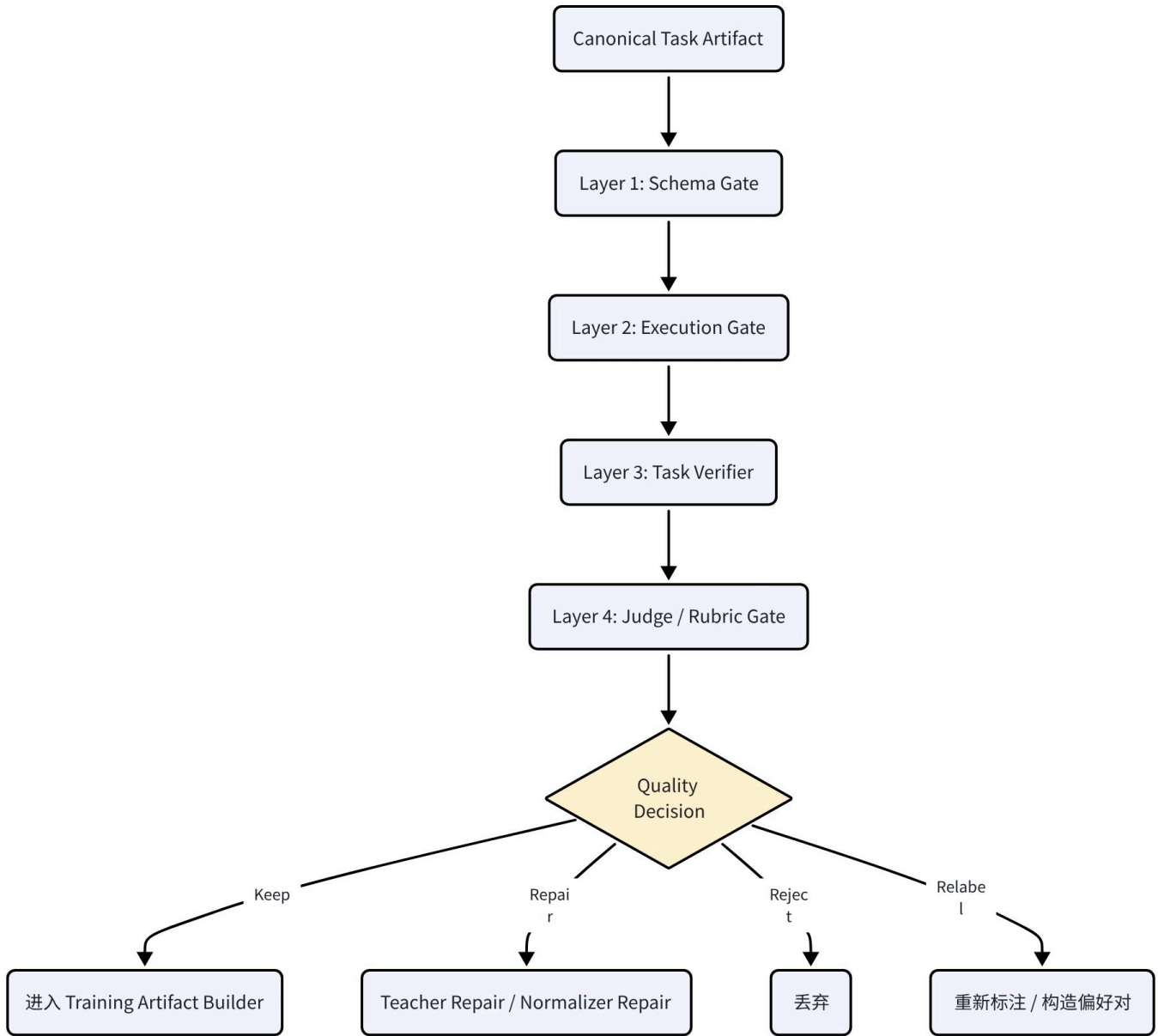
IR	建议字段	QualityGate 重点
PatchIR	files、diff、new_files、deleted_files、run_command、tests、expected_behavior。	patch 是否能 apply，代码能否编译，测试是否通过，失败日志是否可定位。
ClaimCitationIR	claims、evidence_spans、citations、source_url、confidence、unsupported_claims。	引用是否存在，claim 是否被证据支持，来源是否可靠，是否有 hallucination。

✅ **实现策略：**第一版不需要把所有 IR 都做满，但接口要先留出来。每个 vertical 至少要定义 contract name、schema version、normalizer、execution env、verifier、artifact builders 六个配置项。

5.6 QualityGate 为什么必须接 ExecutionEnv / Renderer / Verifier

❗ **关键判断：**没有环境的 QualityGate 只能判断“像不像对”；有环境的 QualityGate 才能判断“是不是真的对”。GUI、PPT、Code、Agent、Research 这类任务尤其不能只靠 LLM Judge。

QualityGate 不是一个打分模型，而是一套任务验证流水线。它会调度 schema checker、renderer、sandbox、emulator、tool executor、symbolic verifier、LLM/VLM judge 等组件。不同任务用不同插件，但输出统一的 Quality Decision。



任务	ExecutionEnv / Renderer	Task Verifier	Judge / Rubric
Math	Python、SymPy、 calculator。	final answer exact match、 numeric tolerance、 symbolic equivalence、 PRM。	步骤清晰度、推理自然性、解 法简洁性。
Code	Code Sandbox、 compiler、pytest、 static analyzer。	编译是否通过，单测是否通 过，安全扫描是否报警。	可读性、可维护性、复杂度、 风格一致性。
PPT	PptxGenJS、python- pptx、PPTX renderer、 slide image converter。	PPTX 能否打开，元素是否越 界/重叠，内容是否覆盖输入。	Content、Design、 Coherence、视觉可读性。

GUI	Android emulator、desktop env、browser env。	click 是否命中 bbox，页面状态是否达成，轨迹是否成功。	轨迹是否高效，动作是否冗余，是否误触。
Tool Agent	Tool Executor、mock tools、真实 API sandbox。	tool arguments 是否合法，observation 是否支持 final answer。	规划合理性、工具选择质量、失败恢复能力。
Deep Research	Browser/Search Env、citation fetcher。	citation 是否可访问，claim 是否被 evidence 支持。	来源可靠性、结构清晰度、论证完整性。

没有环境时

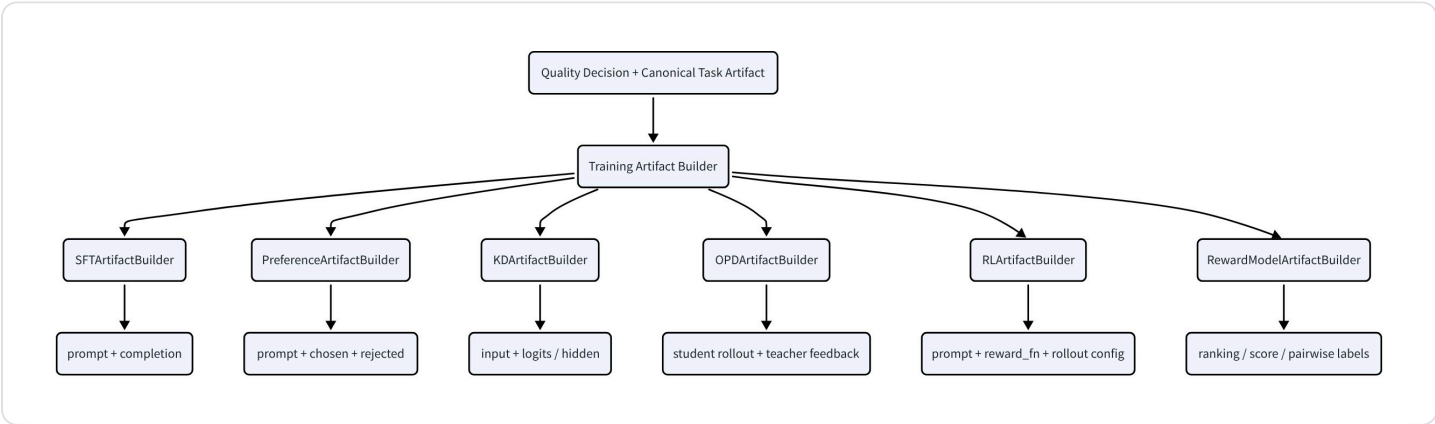
- 只能看文本是否像正确答案。
- 很难发现 PPT 元素重叠、代码跑不通、GUI 点错位置。
- LLM Judge 容易被流畅表达误导。
- 失败样本很难做精确归因。

接入环境后

- 可以直接验证任务是否成功。
- 能生成结构化错误：compile_error、layout_overlap、bbox_miss。
- 能把失败转成 repair data 或 chosen/rejected pair。
- 能把线上失败样本稳定回流到飞轮。

5.7 Training Artifact Builder：从任务产物编译到训练数据

Training Artifact Builder 只做一件事：根据 recipe，把通过 QualityGate 的任务级产物、失败样本、teacher feedback、verifier result 编译成训练框架能吃的数据格式。它不负责判断数据好不好，也不负责执行训练。



Builder	输入	输出	典型 Recipe
SFTArtifactBuilder	高质量 teacher output、repair output、trace。	prompt + completion 或 messages。	SFT、Trace SFT、Response KD。

PreferenceArtifact Builder	student bad output、teacher repaired output、judge ranking。	prompt + chosen + rejected 。	DPO、SimPO、KTO、Reward Model。
KDArtifactBuilder	teacher logits、top-k logprobs、hidden states。	input_ids + teacher distribution / hidden path。	Logit KD、Hidden KD、MiniLLM、DistiLLM。
OPDArtifactBuilder	student rollout 和 teacher 在 rollout 上的反馈。	rollout + teacher_feedback + token/step targets。	GKD、OPD、SOD。
RLArtifactBuilder	prompt set、verifier、reward function、environment config。	prompt + reward_fn + rollout_config。	GRPO、PPO、RLVR。
RewardModelArtifactBuilder	ranking、score、pairwise labels、rubric score。	score labels、pairwise labels、ranking list。	Reward Model、reranker、rubric reward。



一句话区分：IR Normalizer 把输出变成“能执行的”；QualityGate 判断“能不能用”；Training Artifact Builder 把它变成“能训练的”。



关键约束：train/eval/test 必须隔离。飞轮新增数据可以进入下一轮训练集或 hard set，但不能污染 frozen benchmark。所有 teacher 输出、repair 输出和 verifier 结果都要记录版本。

6. 数据增强模块：不是改写，而是可验证的新样本生成

现代数据增强不是把少量 seed 题换个说法，而是从 seed、任务定义、原始资料和失败样本中抽取能力点，生成新的可验证样本，再用难度、多样性和可靠性筛选。

Seed / Source / Failure



能力点抽取



概念图 / 题型图 / 任务图



增强模式	核心思想	适用场景	参考方向
seed_concept_expand	从 seed 里抽 topic、knowledge point、题型，再构建概念图生成新题。	只有 10-100 个数学题、代码题、结构化任务样例。	MathScale
solution_backtranslation	从解法结构反推新问题，而不是从问题 paraphrase 问题。	数学推理、代码修复、可分解步骤的任务。	MathGenie
source_grounded_generation	从教材、网页、论文、代码仓库、业务文档中抽任务和答案。	seed 少但有外部资料的垂直任务。	MAmmoTH2、WebR
difficulty_hiking	把简单题逐步加约束、加步骤、混合概念，形成更难样本。	需要拉高推理难度或构造 hard set。	SAND-Math、SPARQ
weakness_focused_generation	根据 student 失败类型，定向生成薄弱概念样本。	飞轮第二轮以后，已有失败样本和错误分类。	MathSmith、failure-driven synthesis
rubric_generation	同时生成评价标准，让 QualityGate 和 DPO 数据构造更稳定。	PPT、Research、GUI、开放式回答。	OpenRubrics、Rubrics as Rewards

分享页里对大模型专项数据的总结，对 DistillWheel 很关键：不同能力的数据生产方式本来就不一样，不能用同一个“扩写 prompt”处理所有任务。

专项能力	数据生产方式	DistillWheel 中的落地模块
代码能力	生成编程题和解法，用 parser、linter、unit tests、执行反馈过滤；失败样本带错误日志让 teacher 自修复。	Data Synthesizer 生成任务；QualityGate 执行测试；Failure Miner 保存 traceback 和 failed test。
数学推理	建立数学技能 taxonomy，生成 step-by-step reasoning trace，用正确答案、outcome reward、stepwise reward 过滤。	TaskSpec 声明 subskills；Verifier 用 exact/SymPy/PRM；Preference Builder 构造 correct vs wrong。
工具调用	生成 function tuple、API schema、user query、API call、API response，多轮场景可由多个 agent 模拟。	IR Schema 对齐 tool schema；TeacherHub 生成 tool trace；Evaluator 检查调用参数和执行结果。
事实性/RAG	拆 claim、找 evidence、校验 citation、过滤 unsupported answer 和 hallucination。	QualityGate 使用 citation verifier、MiniCheck/ARES 类 factuality evaluator。
PPT/GUI		

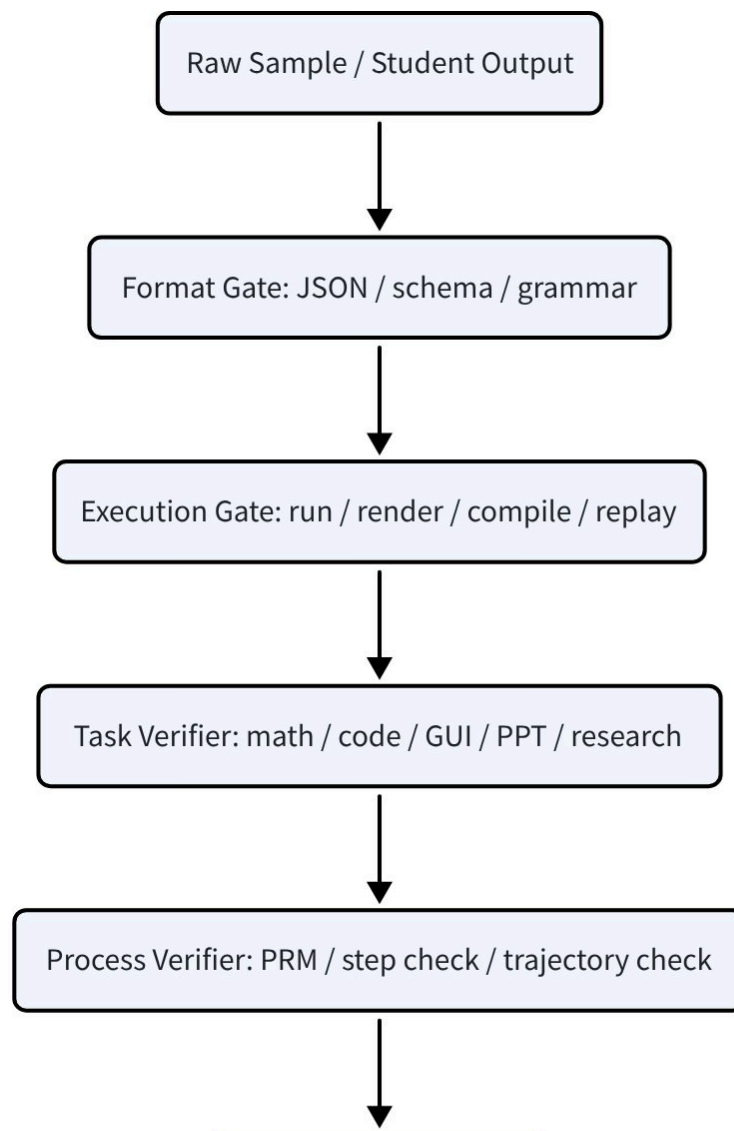
PPT 走 IR -> renderer -> layout verifier;
GUI 走 action/bbox/trajectory ->
environment replay。

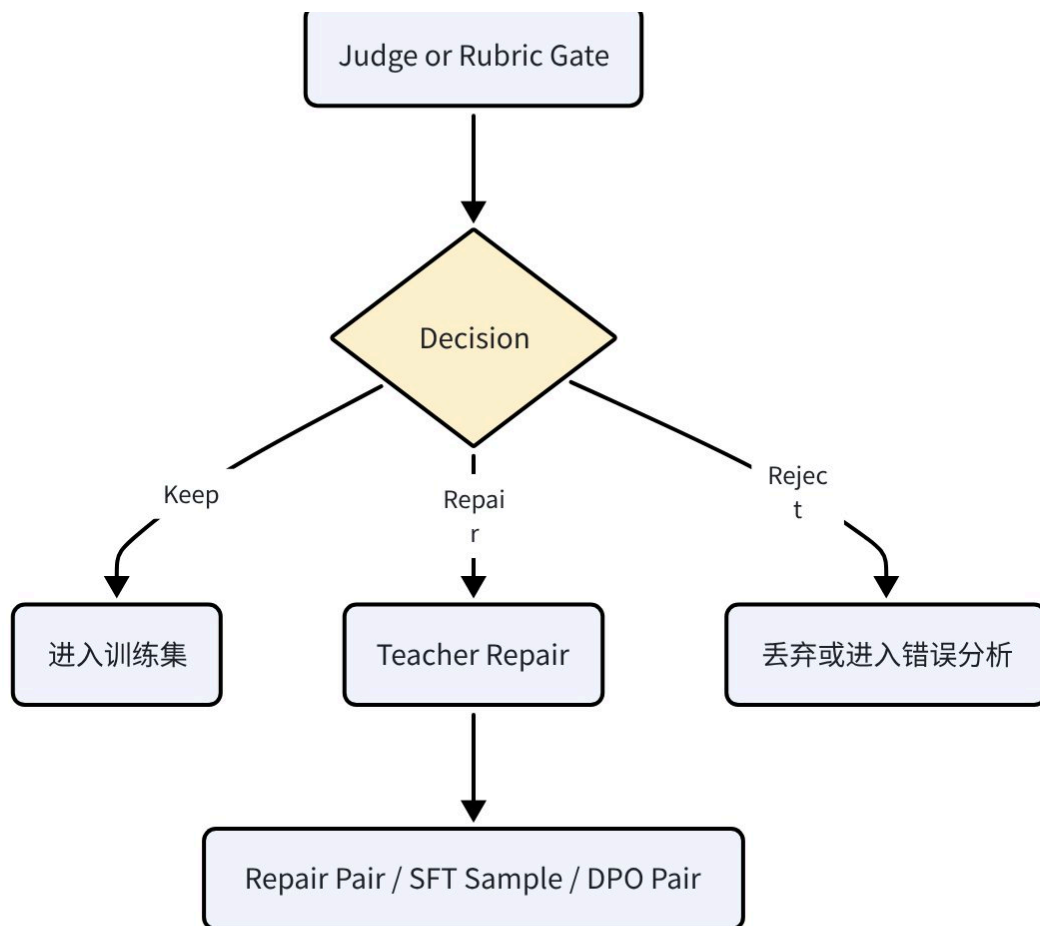
IR Schema、Domain Evaluator 和 Failure
Miner 必须任务专属实现。

✓ **MVP 做法：**第一版不追求一次集成所有生成策略。优先实现
source_grounded_generation、seed_concept_expand 和 failure_repair_generation。这
样既能冷启动，也能在第一轮 student 出错后形成闭环。

7. QualityGate 与纠错：任务专属 Verifier，而不是统一 Judge

不同能力的“正确”定义不同，因此纠错不能只靠一个 LLM Judge。DistillWheel 把纠错抽象成 Verifier + Repairer 插件：Verifier 负责判断哪里错，Repairer 负责让 teacher 修复错误并产出可训练样本。





能力	Verifier	Repairer	训练数据产物
Math	exact match、数值 tolerance、SymPy 等价、PRM、ThinkPRM、Lean theorem prover。	teacher 根据错误步骤重写解法，必要时生成同概念新题。	answer SFT、trace SFT、step-level reward、hard set。
Code	语法检查、编译、单元测试、隐藏测试、静态分析、CodeQL、KLEE。	teacher 根据 traceback、failed test、安全告警修复代码。	bug-fix SFT、execution feedback SFT、DPO pair。
PPT	JSON Schema、PptxGenJS/python-pptx renderer、layout overlap、图表数据一致性、VLM judge。	teacher 修复 Semantic IR、Render IR 或生成代码。	IR SFT、repair pair、layout hard negative。
Research	claim decomposition、citation checking、source availability、faithfulness evaluator。	teacher 重新找证据、修 claim、补引用。	grounded answer SFT、citation preference pair。
GUI Agent	bbox hit、click accuracy、environment state、trajectory replay、task success。	teacher 重新规划 action 或修复 trajectory。	action SFT、trajectory SFT、step-level preference。

```

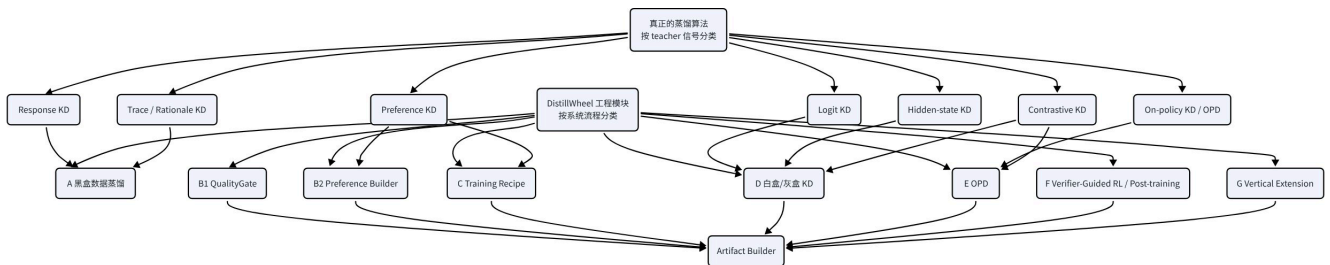
1 class Verifier:
2     def verify(self, task_input, model_output) -> VerificationResult:
3         ...
4
5 class Repairer:
6     def repair(self, task_input, bad_output, errors) -> RepairedOutput:
7         ...

```

8. 蒸馏算法层：算法、模块、Recipe 分层

这里必须分清三层：蒸馏信号 / 数据形态回答“teacher 输出了什么”；训练 recipe 回答“用什么目标函数训练”；Trainer Backend 回答“用哪个框架把 recipe 跑起来”。DistillWheel 的工程表不是严格蒸馏算法表，而是把算法、artifact、verifier 和训练后端组织起来的系统分类。

本节最终口径：真正的蒸馏算法按 teacher 信号分类；DistillWheel 的 A-G 表按工程模块和训练流程分类；GRPO/RLVR 属于 verifier-guided post-training，通常不是严格 KD；只有显式使用 teacher supervision、teacher repair、teacher judge 或 teacher-guided process feedback 时，才构成 hybrid distillation。



8.1 严格蒸馏算法 taxonomy

蒸馏算法族	Teacher 信号	代表方法	本质
Response KD	teacher final output	Sequence KD、API teacher SFT、R1-style distill	学习 teacher 的最终输出。
Trace / Rationale KD	teacher reasoning trace、CoT、plan	Distilling Step-by-Step、R1-style reasoning trace distill	学习 teacher 的中间推理过程。

Logit KD	teacher logits、probability distribution	Hinton KD、Forward KL、Reverse KL、MiniLLM、DistiLLM	学习 teacher 的概率分布。
Hidden-state / Representation KD	teacher hidden states、attention、intermediate representation	TinyBERT、MiniLM 类方法	学习 teacher 的内部表示。
Preference KD	teacher preference、judge score、ranking	teacher-as-judge + DPO / RM / RLAIIF	学习 teacher 对好坏输出的偏好。
On-policy KD / OPD	student rollout 上的 teacher feedback	GKD、SOD、GAD、OPSD	student 先自己生成，再让 teacher 在 student 分布上指导。
Contrastive KD	teacher response vs student response	DistiLLM-2	提高 teacher response 概率，同时降低 student bad response 概率。

8.2 DistillWheel 工程模块 taxonomy

类别	严格 KD?	原因
A. 黑盒数据蒸馏	是，广义 KD	API teacher / open teacher 提供 final answer、CoT、trace、repair 等文本或结构化输出，本质是 response / trace KD。
B1. QualityGate	不是	负责判断数据好不好、能否进入训练集，是质量控制模块。
B2. Preference Builder	不是算法	把好坏输出构造成 chosen/rejected、ranking、score，服务 DPO、SimPO、RM 等。
C. 基础后训练 recipe	不是	SFT、LoRA、DPO、KTO、SimPO 是训练容器。只有训练数据来自 teacher、verifier 或 repair 时，才属于蒸馏系统链路。
D. 白盒/灰盒分布蒸馏	是，狭义 KD	logits、top-k logprobs、hidden states 是传统 KD 的核心 teacher 信号。
E. On-policy 蒸馏	是，前沿 KD	student rollout 上的 teacher feedback 是 OPD 核心，可缓解只学离线 teacher 数据的 exposure bias。
F. Verifier-Guided RL / Post-training	通常不是	GRPO、RLVR 主要依赖 verifier reward。若同时使用 teacher supervision，才是 hybrid KD + RL。
	不是算法族	

G. 多模态 / Agent Vertical Extension	多模态、Agent、GUI 是任务形态；内部可使用 A-F 的算法或 recipe。
-----------------------------------	--

8.3 Artifact 类型

Artifact 类型	长什么样	给谁用
SFT Artifact	prompt、completion；也可以带 reasoning trace。	SFT / Trace SFT。
Preference Artifact	prompt、chosen、rejected；或 ranking、score。	DPO / SimPO / KTO / RM。
KD Artifact	input、teacher logits、top-k logprobs、hidden states。	Logit KD / Hidden KD / DistiLLM。
OPD Artifact	prompt、student rollout、teacher feedback、token/step-level targets。	GKD / OPD / SOD。
RL Artifact	prompt set、reward function、verifier、environment、rollout config。	GRPO / PPO / RLVR。
Reward Model Artifact	ranking、scores、pairwise labels、rubric labels。	Reward Model Training。

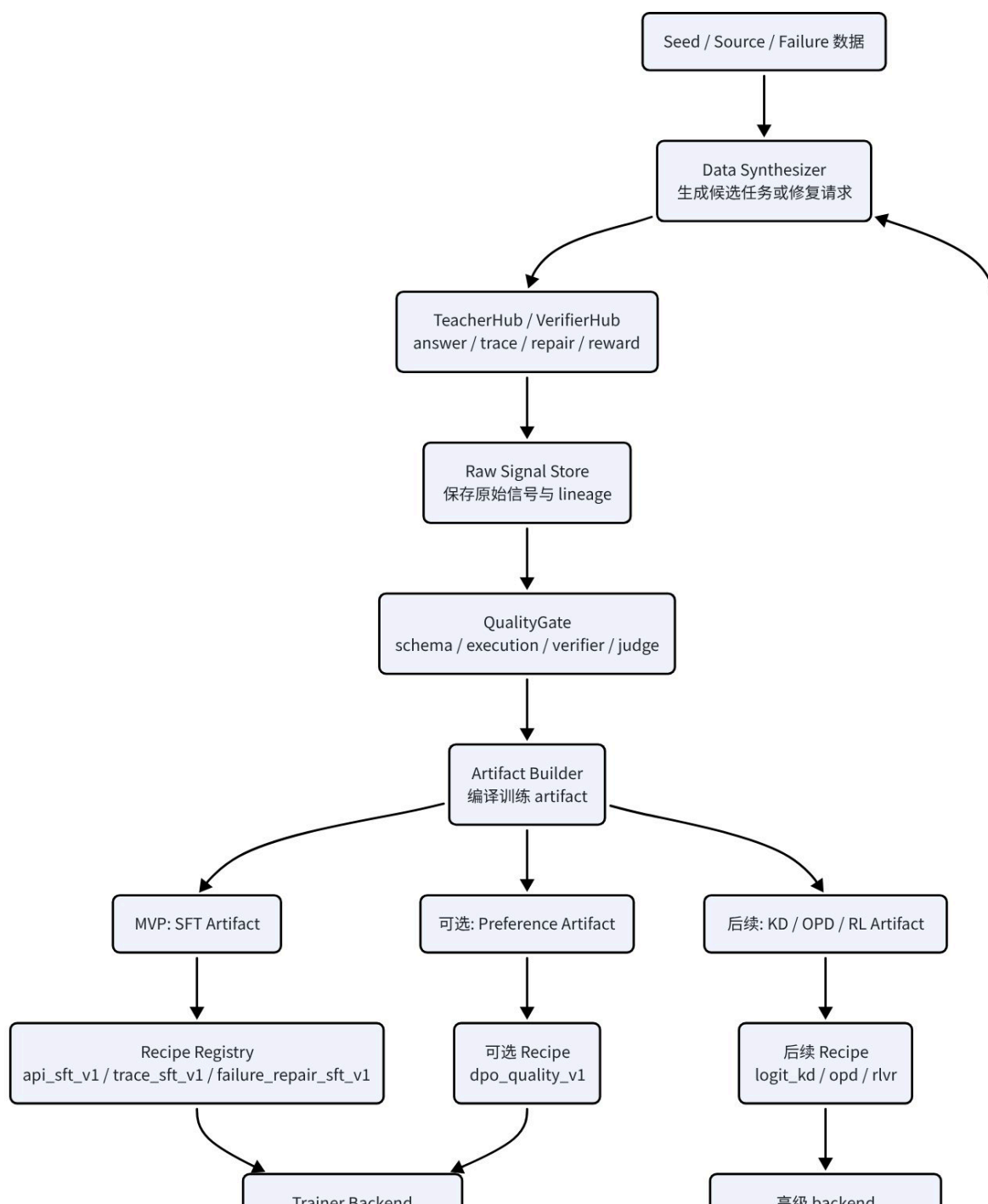
8.4 Recipe 与 Backend 的关系

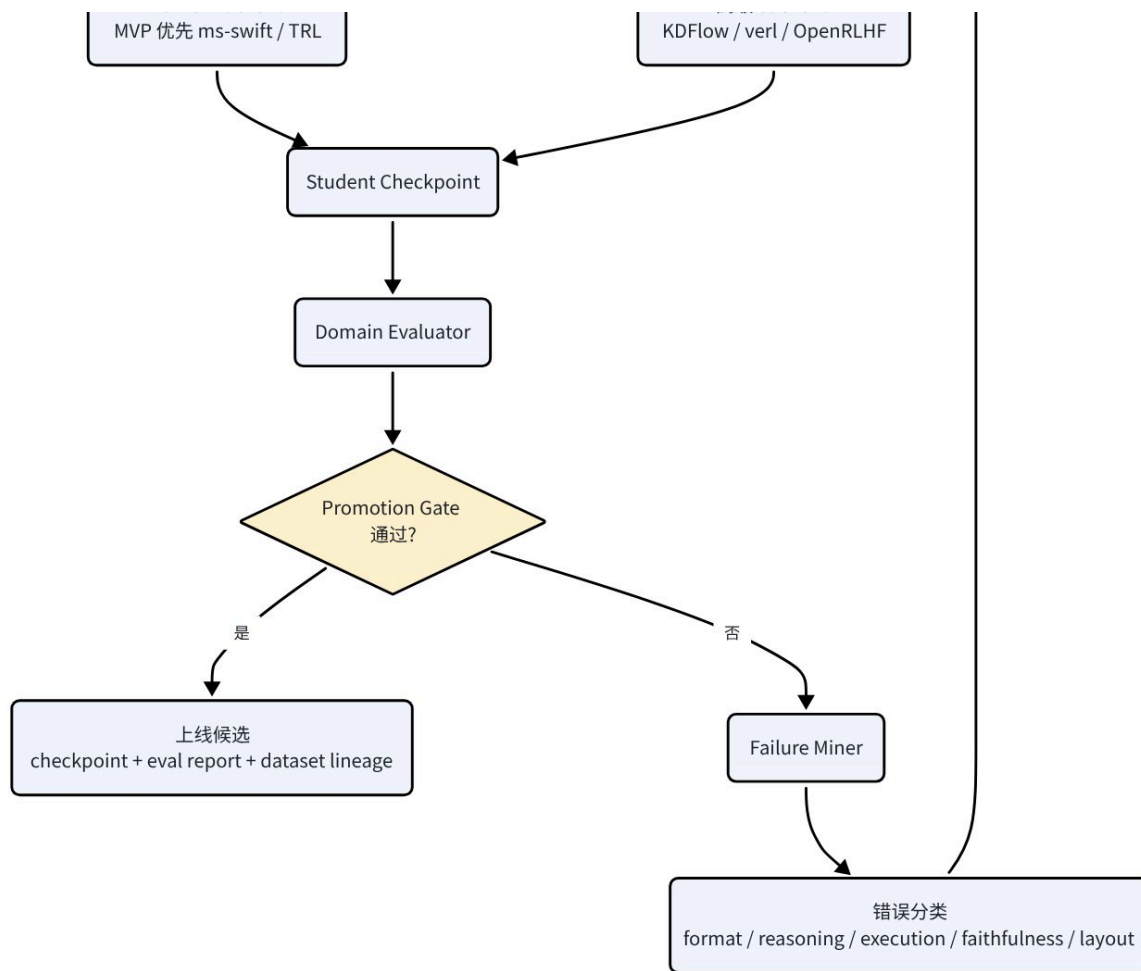
Recipe	输入 artifact	是否 KD	适合 backend
SFT	SFT Artifact	不一定	ms-swift / TRL
Trace SFT	SFT Artifact with trace	是，Trace KD	ms-swift / TRL
DPO	Preference Artifact	不一定	ms-swift / TRL
Logit KD	KD Artifact	是	KDFlow / TRL / EasyDistill
Hidden KD	KD Artifact	是	KDFlow / 自研
GKD / OPD / SOD	OPD Artifact + step info	是	verl / KDFlow / TRL / 自研
GRPO / PPO / RLVR	RL Artifact	通常不是	verl / OpenRLHF / ms-swift
KDRL	KD Artifact + RL Artifact	是，hybrid	verl / TRL / 自研

! **关键区分：**OPD 是蒸馏 recipe，因为它需要 teacher 在 student rollout 上给 feedback；GRPO、PPO、RLVR 是 RL recipe，通常只需要 verifier reward，不一定有 teacher。Trainer Backend 只负责执行 recipe，不决定这个方法是不是 KD。

9. 数据飞轮 MVP

数据飞轮 MVP 的目标不是一次性覆盖所有 KD/RL 算法，而是证明一条最小链路能闭环：student 失败后，verifier 定位错误，teacher 修复或补充信号，Artifact Builder 生成可训练 artifact，Recipe Registry 选择最小 recipe 重新训练，领域评测验证是否提升。

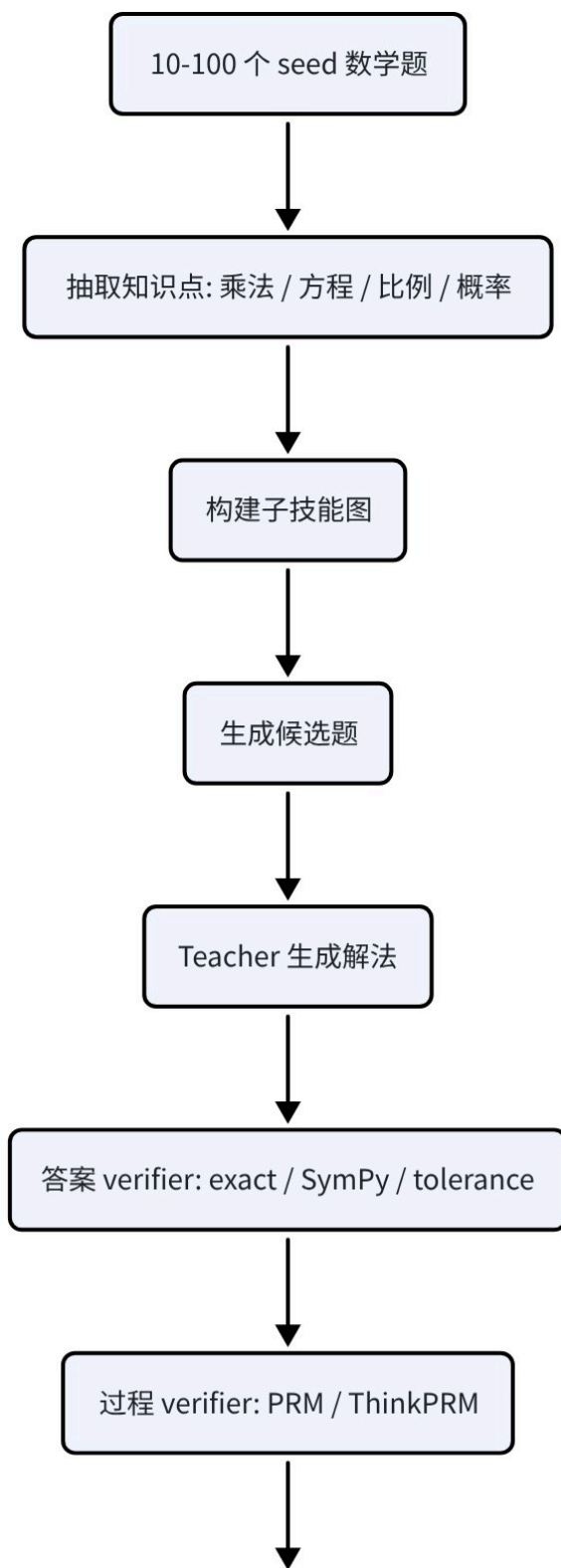


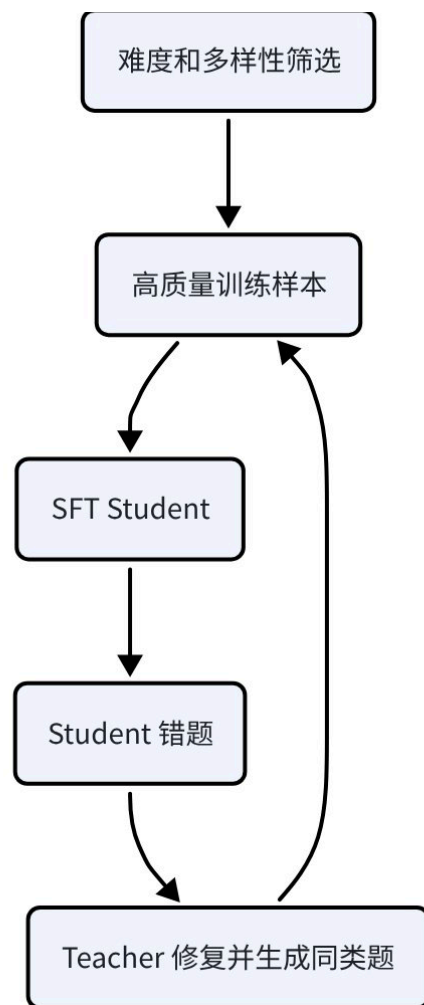


MVP 环节	要实现的最小能力	验收标准
TaskSpec	可以用 task.yaml 描述一个 vertical。	同一配置可复跑，产出相同目录结构和版本号。
TeacherHub / VerifierHub	至少支持一种 teacher 和一个任务 verifier。	可生成 answer、trace、repair 或 reward，并写入 Raw Signal Store。
QualityGate	至少支持 schema gate、execution/render gate、LLM judge 或任务 verifier。	每条样本都有 keep/repair/reject 和错误原因。
Artifact Builder	MVP 先产出 SFT Artifact；Preference Artifact 作为可选。	同一批 raw signals 可稳定编译成训练数据，并记录 artifact_version。
Recipe Registry	先支持 api_sft_v1、trace_sft_v1、failure_repair_sft_v1。	recipe 能声明 required_artifacts、backend、训练参数和 eval。
Trainer	接 ms-swift 或 TRL 做 SFT；DPO 可作为可选。	自动生成训练配置，输出 LoRA/checkpoint。
Evaluator	领域指标 + hard set + 成本/延迟指标。	能比较 base student、distilled student、teacher。

10. 典型任务如何落地

10.1 10-100 个数学题扩成训练集





数学任务的关键不是生成多少题，而是每题是否覆盖明确能力点，答案是否可验证，过程是否能定位错误。最终数据集中应该同时有 final answer、step trace、verifier result 和 difficulty metadata。

10.2 API teacher 蒸馏数学推理能力

如果 teacher 只是商业 API，大多数情况下只能做黑盒蒸馏：让 API teacher 产出题目解法、最终答案、纠错讲解、偏好判断，再训练 student 模仿这些输出。不能默认做 full-vocab logits KD、hidden-state KD 或 attention KD。

用户创建任务 math_reasoning



配置 API teachers



配置 student 小模型



准备 seed math problems

✓ API teacher 能做:

- 生成 final answer、step-by-step trace、讲解和修复。
- 作为 judge 产生 chosen/rejected 或质量评分。
- 对 student 错题做 hard case relabel 和 repair。
- 生成同知识点、不同难度的新题。

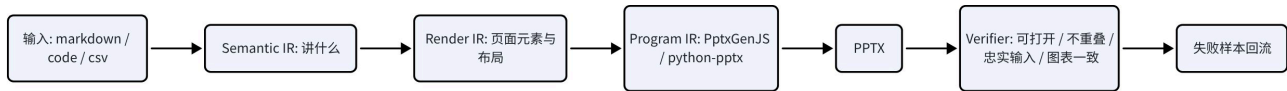
💡 API teacher 通常不能做:

- 完整 vocabulary logits KD。
- hidden-state / attention KD。
- pruning + KD 这类需要权重访问的方法。
- 绕过 ToS 把闭源 API 输出直接训练竞品模型。

math_reasoning task.yaml

```
1  task_name: math_reasoning
2  version: v0.1
3  capability:
4    name: mathematical_reasoning
5    subskills:
6      - arithmetic_word_problem
7      - algebra
8      - equation_solving
9      - geometry
10     - probability
11  output_contract:
12    type: math_solution_trace
13    fields:
14      - problem
15      - reasoning_steps
16      - final_answer
17  quality_gates:
18    - final_answer_parse
19    - exact_match
20    - symbolic_equivalence
21    - numeric_tolerance
22    - process_reward_model
23  recipes:
24    - api_trace_sft_v1
25    - failure_repair_sft_v1
26    - dpo_correct_vs_wrong_v1
```

10.3 Code2PPT



Code2PPT 不建议让模型直接输出 PPTX。模型应输出可校验 IR，再由 deterministic renderer 生成最终文件。这样失败可以归因到 schema、内容、布局、渲染或图表数据，而不是只得到一个“生成坏了”的结论。

10.4 Code 任务

代码任务的 verifier 最清晰：编译、运行测试、静态分析、安全扫描。失败日志本身就是高质量 teacher repair 输入。DistillWheel 可以把 failed test、traceback、CodeQL warning、KLEE counterexample 结构化存储，再生成 SFT 或 DPO 数据。

11. 与已有项目的关系

项目	定位	DistillWheel 的处理方式
EasyDistill	通用 LLM 蒸馏工具包，覆盖 black-box/white-box KD、数据合成、SFT、ranking、RL。	作为竞品和 recipe 参考，不作为第一版 Trainer 同层后端强依赖。
KDFlow	高效 KD runtime，面向白盒/灰盒 KD、off-policy/on-policy、cross-tokenizer、多模态 KD。	作为后续白盒 KD/OPD 算法后端参考；MVP 不依赖。
ms-swift	后训练执行框架，适合 SFT、DPO、GRPO、多模态训练。	MVP 默认训练后端。
TRL	Hugging Face post-training 框架，支持 SFT、DPO、GKD、GRPO 等。	用于后续自定义 loss 和 GKD recipe。
verl	大规模 RL post-training 框架，适合 RLVR、GRPO、Agentic RL。	有稳定 verifier 后接入。
NVIDIA Data Flywheel	生产日志 -> 数据集 -> 微调 -> 评测 -> 迭代的工程蓝图。	作为飞轮闭环参考，但 DistillWheel 更强调垂直 TaskSpec、IR 和 verifier 插件。

<https://github.com/modelscope/easydistill>

EasyDistill 仓库

<https://github.com/songmzhang/KDFlow>

KDFlow 仓库

<https://github.com/NVIDIA-AI-Blueprints/data-flywheel>

NVIDIA Data Flywheel

12. CLI、产物和目录形态

分享页里提到的 DistillWheel 命名和 CLI 思路可以直接转成工程形态：DistillWheel 的 CLI 不应该只是 trainer wrapper，而应该围绕一次飞轮 run 的生命周期组织命令。

推荐 CLI

distillwheel CLI

```
1 dw init math_reasoning
2 dw synthesize --task task.yaml
3 dw gate --run runs/001
4 dw train --recipe
  api_trace_sft_v1
5 dw eval --checkpoint
  outputs/student
6 dw mine --eval-report
  reports/eval.json
7 dw spin --from-failures
  runs/001/failures
```

一次 run 的产物

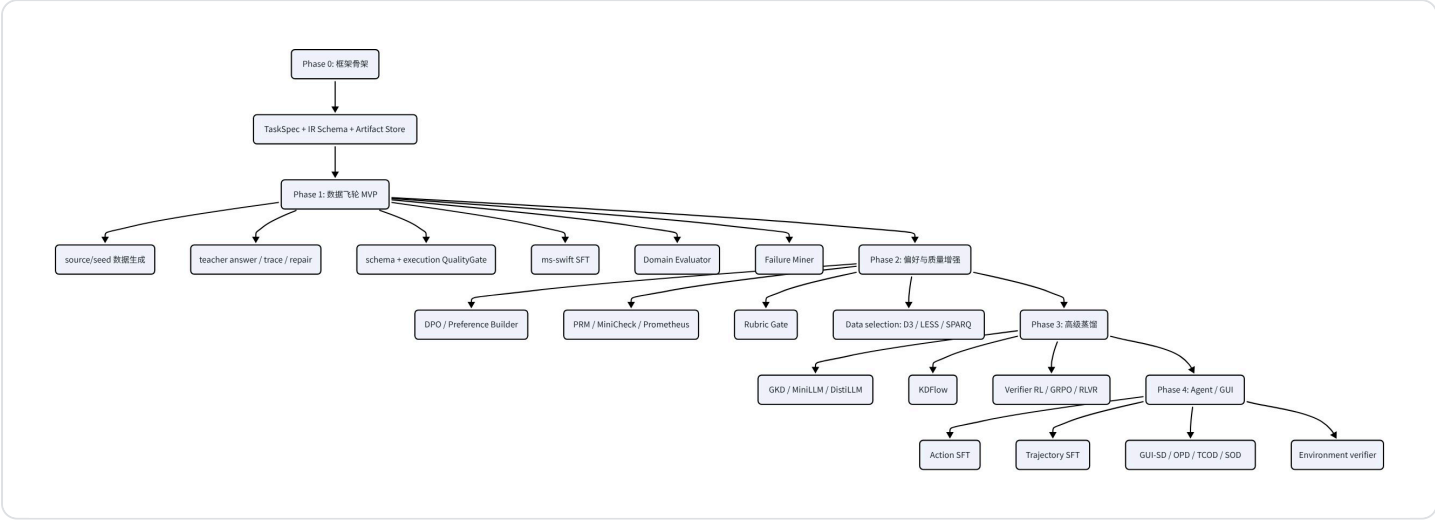
artifact layout

```
1 runs/001/
2   task.yaml
3   source/
4   raw_teacher_outputs/
5   quality_gate_results/
6   datasets/
7     sft.jsonl
8     preference.jsonl
9   training/
10     swift_config.yaml
11     logs/
12     checkpoints/
13     eval/
14       report.json
15       hard_cases.jsonl
16     failures/
17       mined_failures.jsonl
```

命令	做什么	输出
<code>dw init</code>	创建垂直任务骨架、TaskSpec、IR Schema 和默认 evaluator。	任务目录和配置模板。
<code>dw synthesize</code>	调用 TeacherHub 做 source-grounded、concept-expand 或 failure-repair 数据生成。	raw teacher outputs 和 trace。

dw gate	执行 schema、verifier、judge、dedup、rubric gate。	keep/repair/reject 决策和错误类型。
dw train	选择 recipe 并调用 ms-swift、TRL 或 verl 后端。	checkpoint、LoRA、训练日志。
dw eval	在 frozen eval、hard set、成本延迟指标上评测 student。	领域评测报告。
dw mine / dw spin	挖掘失败样本并启动下一轮 teacher repair。	failure set 和下一轮训练数据。

13. 工程实现路线



阶段	交付物	判断是否完成
Phase 0	任务配置、样本格式、数据目录、run_id、artifact lineage。	能用同一 task.yaml 复现实验目录和数据版本。
Phase 1	一个垂直 case 的黑盒蒸馏闭环。	student 在 frozen eval 和 hard set 上比 base 有提升。
Phase 2	DPO 数据构造、PRM/Judge、rubric gate、数据选择。	失败样本复现率下降，数据效率提升。
Phase 3	GKD、DistiLLM、KDFlow、RLVR recipe。	在 open-weight teacher 或 verifier 任务上超过黑盒 SFT baseline。
Phase 4	Agent/GUI 的 action、trajectory、环境 verifier 和 OPD 类训练。	GUI grounding 或 agent benchmark 上形成可复现闭环。

14. 关键指标

指标	含义	使用位置
teacher parity	student 达到 teacher 质量的比例。	Promotion Gate
task success rate	领域任务是否通过 verifier，例如代码测试、PPT 渲染、数学答案。	Domain Evaluator
failure recurrence	同类错误在下一轮是否减少。	Failure Miner
data efficiency	每新增 1K 样本带来的指标提升。	Data Synthesizer
quality pass rate	候选样本通过 QualityGate 的比例。	QualityGate
cost / latency reduction	student 相对 teacher 的成本和延迟下降。	Promotion Gate

 **最终形态：**DistillWheel 的核心价值是把每个 vertical 做成可复用闭环：TaskSpec 定义任务，Data Synthesizer 生成可验证数据，QualityGate 过滤和纠错，Recipe Registry 训练 student，Domain Evaluator 证明效果，Failure Miner 驱动下一轮数据。

(注：内容由 AI 生成，请谨慎参考)